DOCUMENTATION TECHNIQUE

Infrastructure VDI — TechVault

Projet	TechVault — Laser-Campus Numérique
Phase	Phase 4 — Automatisation VDI (Semaine 8)
Version	1.0
Date	20 mars 2026
Auteurs	Alexandru NASUI, Benson FRANCOIS

1. Introduction

Ce document constitue la documentation technique de la phase 4 du projet TechVault, portant sur la mise en œuvre d'une solution de postes de travail virtuels (VDI — Virtual Desktop Infrastructure). Il couvre l'ensemble de la chaîne d'automatisation : de la demande utilisateur via l'interface Web Flask jusqu'au poste de travail opérationnel, en passant par le provisionnement OpenTofu et la configuration post-déploiement Ansible.

1.1 Contexte

Laser-Campus Numérique est un centre de formation dispensant des cours en présentiel et à distance. L'objectif est de fournir à chaque apprenant un poste de travail virtuel Debian avec environnement graphique Cinnamon, créé automatiquement à la demande, intégré à l'annuaire central LDAP/Kerberos et aux partages NFS.

1.2 Périmètre

Ce document couvre le projet n°2 : Virtualisation et automatisation des postes de travail. Il s'appuie sur les services déployés lors du projet n°1 (LDAP, Kerberos, NFS, DNS).

1.3 Technologies utilisées

Technologie	Rôle	Version
Proxmox VE	Hyperviseur bare metal	9.x
OpenTofu	Infrastructure as Code — provisionnement VM	≥ 1.6
Ansible	Configuration automatisée post-déploiement	≥ 2.15
Flask	Interface Web de commande de VM	3.x
Nginx	Reverse proxy HTTPS	Dernière stable
cloud-init	Initialisation des VMs au premier démarrage	Intégré Proxmox
OpenLDAP	Annuaire centralisé des utilisateurs	2.6
Kerberos (MIT)	Authentification sécurisée	1.20+
NFS v4	Stockage réseau partagé	Kernel Linux
Wazuh	Agent de supervision/sécurité	Dernière stable

2. Architecture VDI

2.1 Vue d'ensemble

L'architecture VDI repose sur un hyperviseur Proxmox VE unique (bare metal) hébergeant l'ensemble des machines virtuelles. Chaque VM VDI est créée à partir d'un template Debian+Cinnamon pré-configuré, cloné et personnalisé automatiquement via cloud-init.

2.2 Adressage réseau

Serveur	IP	Rôle
Proxmox (hyperviseur)	192.168.1.200	Hôte de virtualisation
Serveur LDAP / Kerberos	172.16.0.118	Annuaire + authentification
Serveur NFS	172.16.0.111	Stockage privé et commun
Serveur DNS	172.16.0.116	Résolution de noms + DNS dynamique
Serveur Flask / Nginx	172.16.0.119	Interface Web de provisionnement
VMs VDI	DHCP — vmbr1 VLAN 5	Postes de travail virtuels

2.3 Flux de création d'une VM

Le processus de création d'une VM VDI suit la séquence suivante :

1. L'utilisateur remplit le formulaire Web Flask (email, mot de passe)
2. Flask calcule le UID/GID LDAP et génère un nom de VM unique à partir du mail
3. Flask lance OpenTofu → clone du template Proxmox, attribution IP DHCP
4. OpenTofu retourne l'IP et le VMID de la VM créée
5. Flask génère l'inventaire Ansible dynamiquement depuis le template inventory.tpl
6. Flask déclenche le playbook Ansible → 9 rôles de configuration
7. Ansible redémarre la VM pour appliquer le hostname
8. Flask envoie un email à l'utilisateur avec login, mot de passe temporaire et accès VNC/SPICE

3. Template de base Proxmox

Toutes les VMs VDI sont créées par clonage d'un template appelé "Modele-Debian-cloudinit". Ce template doit être préparé une seule fois sur Proxmox.

3.1 Caractéristiques du template

Paramètre	Valeur
Nom	Modele-Debian-cloudinit
OS	Debian 13 (trixie)
Environnement graphique	Cinnamon
CPU	2 vCPU (type host)
RAM	4096 Mo
Disque	32 Go — scsi0 — local-lvm
Réseau	virtio — bridge vmbr1 — VLAN tag 5
Affichage	QXL 64 Mo (compatible SPICE)
VMID base VDI	2000 + UID utilisateur
Accès distant	VNC / SPICE

3.2 Prérequis installés dans le template

Les paquets suivants doivent être installés et activés dans le template :

- cloud-init
- qemu-guest-agent
- openssh-server
- python3
- sudo
- curl

3.3 Conversion en template

- Sur le nœud Proxmox, après installation et nettoyage :
- cloud-init clean
- rm -f /etc/machine-id
- truncate -s 0 /etc/machine-id
- # Dans l'interface Proxmox : clic droit sur la VM → Convert to Template

4. Infrastructure as Code — OpenTofu

OpenTofu (fork open-source de Terraform) est utilisé pour provisionner les VMs VDI sur Proxmox via son API REST. Il s'appuie sur le provider telmate/proxmox version 3.0.2-rc07.

4.1 Structure des fichiers

```
opentofu/

➔  provider.tf        (Déclaration du provider Proxmox)
➔  variables.tf       (Déclaration de toutes les variables)
➔  terraform.tfvars   (Valeurs des variables)
➔  main.tf            ( Ressource principale : proxmox_vm_qemu)
➔  outputs.tf         (Sorties : IP et VMID de la VM créée)
➔  .terraform.lock.hcl  (Verrouillage des versions)
➔  terraform.tfstate    ( État courant )
```

4.2 Variables disponibles

Variable	Type	Description	Valeur par défaut
api	string	URL API Proxmox	https://172.16.10.10:8006/api2/json
token_id	string (sensitive)	Identifiant token API	—
token_secret	string (sensitive)	Secret token API	—
node	string	Nœud Proxmox cible	pve
storage	string	Stockage des disques VM	local-lvm
modele	string	Nom du template à cloner	Modele-Debian-cloudinit
nombre	number	Nombre de VMs à créer	1
vm_name	string	Nom de la VM	—
vm_user	string	Utilisateur cloud-init	—
vm_uid	string	UID LDAP de l'utilisateur	—
vm_gid	string	GID LDAP de l'utilisateur	—

4.3 Ressource principale (main.tf)

La ressource proxmox_vm_qemu crée la VM avec les paramètres suivants :

```
resource "proxmox_vm_qemu" "student_vm" {
  vmid        = 2000 + tonumber(var.vm_uid)
  name        = var.vm_name
  target_node = var.node
  clone       = var.modele
  full_clone  = true
  memory      = 2048
  boot        = "order=scsi0"
  ipconfig0   = "ip=dhcp"
  ciuser      = var.vm_user

  cpu   { cores = 2 ; sockets = 1 ; type = "host" }
  vga   { type = "qxl" ; memory = 64 }
```

```
 disk {
   slot    = "scsi0"
   storage = var.storage
   size    = "32G"
   discard = true
 }

 network {
   id     = 0
   model  = "virtio"
   bridge = "vmbr1"
   tag    = 5
 }
}
lifecycle {
   ignore_changes = [network, cipassword, disk, bootdisk]
 }
}
```

4.4 Sorties (outputs.tf)

Après provisionnement, OpenTofu expose deux sorties utilisées par Flask :

```
output "vm_ip"  { value = proxmox_vm_qemu.student_vm.default_ipv4_address }

output "vm_id"  { value = proxmox_vm_qemu.student_vm.vmid }
```

4.5 Exécution

```
- Initialisation (première fois)

tofu init
```

```
-       Vérification du plan
```

```
tofu plan -var="vm_name=vm-johndoe" -var="vm_user=johndoe" \

        -var="vm_uid=10042" -var="vm_gid=5001"

 - Déploiement

tofu apply -auto-approve \

         -var="vm_name=vm-johndoe" -var="vm_user=johndoe" \

         -var="vm_uid=10042" -var="vm_gid=5001"

 -  Suppression

tofu destroy -auto-approve
```

5. Configuration automatisée — Ansible

Ansible prend le relais après OpenTofu pour configurer la VM fraîchement créée. Il s'exécute depuis le serveur Flask (utilisateur www-data) via SSH, en utilisant une clé RSA dédiée.

5.1 Structure des fichiers

```
ansible/

➔ ansible.cfg        (Configuration globale Ansible)
➔ inventory.ini      ( Inventaire statique)
➔ inventory.tpl      (Template d'inventaire dynamique)
➔ playbook.yml       (Playbook principal VDI)
```

5.2 Configuration (ansible.cfg)

Paramètre	Valeur	Description
remote_user	ais	Utilisateur SSH sur les VMs
private_key_file	/var/lib/www-data/.ssh/id_rsa	Clé SSH de l'utilisateur www-data
host_key_checking	False	Désactivé pour VMs fraîches
become_method	sudo	Élévation de privilèges
local_tmp	/var/lib/www-data/.ansible/tmp	Répertoire tmp local www-data
remote_tmp	/tmp/.ansible	Répertoire tmp distant

5.3 Inventaire dynamique (inventory.tpl)

Le template d'inventaire est généré dynamiquement par Flask à chaque création de VM, en substituant le nom et l'IP de la VM fraîchement créée par OpenTofu :

```
[vdi]
${vm_name} ansible_host=${vm_ip}

[vdi:vars]
ansible_user=ais
ansible_ssh_private_key_file=~/.ssh/id_rsa
ansible_ssh_extra_args='-o StrictHostKeyChecking=no'
ansible_python_interpreter=/usr/bin/python3
nfs_server=172.16.0.111
ldap_server=172.16.0.118
kerberos_server=172.16.0.118
dns_server=172.16.0.116
domain=techvault.fr
ldap_base_dn=dc=techvault,dc=fr
kerberos_realm=TECHVAULT.FR
```

5.4 Rôles du playbook

Le playbook applique 9 rôles dans l'ordre suivant sur le groupe vdi :

Ordre	Rôle	Action réalisée
1	set_hostname	Applique le nom de la VM (vm-nomutilisateur)
2	dns_client	Configure /etc/resolv.conf vers 172.16.0.116
3	dns_register	Enregistre la VM dans le DNS dynamique
4	ldap_client	Configure sssd / pam pour OpenLDAP
5	kerberos_client	Configure /etc/krb5.conf pour TECHVAULT.FR
6	kerberos_keytab	Génère le keytab machine pour NFS sec=krb5
7	nfs_client	Monte /mnt/prive et /mnt/commun
8	create_user	Crée le compte utilisateur local
9	wazuh-agent	Installe et configure l'agent Wazuh (SIEM)

5.5 Tâches post-rôles

Après l'exécution des rôles, le playbook réalise directement :

- Installation et activation de openssh-server
- Sécurisation SSH : PubkeyAuthentication yes / PasswordAuthentication no
- Flush des handlers (redémarrage SSH)
- Reboot de la VM pour appliquer le hostname définitivement

6. Interface Web Flask

L'interface Web Flask sert de point d'entrée unique pour les utilisateurs. Elle orchestre l'ensemble du cycle de vie : authentification LDAP, création de compte, accès invité et déclenchement du provisionnement VDI.

6.1 Routes disponibles

Route	Méthode	Description
/	GET	Page d'accueil
/login	GET/POST	Authentification via LDAP
/signup	GET/POST	Création de compte étudiant (LDAP + Kerberos + NFS)
/guest	GET/POST	Accès invité temporaire (7 jours)
/dashboard	GET	Tableau de bord utilisateur
/logout	GET	Déconnexion et suppression de session

6.2 Services intégrés

Service	Adresse	Fonction
OpenLDAP	ldaps://ldap.techvault.fr	Authentification + création de compte
Kerberos	kerberos.techvault.fr	Création du principal utilisateur
NFS	nfs.techvault.fr	Création du répertoire privé via SSH
OpenTofu	localhost (subprocess)	Provisionnement VM Proxmox
Ansible	localhost (subprocess)	Configuration post-déploiement

6.3 Création de compte — Flux détaillé

```
POST /signup

➔ Vérification format email
➔ Récupération prochain UID disponible (LDAP)
➔ Création entrée LDAP
➔ Ajout au groupe cn=etudiants
➔ Création principal Kerberos (kadmin distant via keytab)
➔ Création dossier NFS privé via SSH (chmod 700, chown uid:gid)
➔ Retour : uid, groupe, confirmation
```

6.4 Compte invité — Spécificités

Le mode invité génère un mot de passe temporaire aléatoire (8 caractères UUID), crée le compte dans le groupe guests (GID 5002) avec une expiration shadowExpire à J+7, sans création de dossier NFS privé (accès au commun uniquement).

6.5 Démarrage du service

```
# Démarrage direct
python3 app.py

# Via systemd
[Unit]
Description=Flask TechVault
After=network.target

[Service]
User=www-data
WorkingDirectory=/var/www/techvault
ExecStart=/usr/bin/python3 app.py
Restart=always

[Install]
WantedBy=multi-user.target
```

7. Intégrations automatiques

7.1 DNS dynamique

Le rôle Ansible dns_register enregistre automatiquement la VM dans le DNS dynamique au moment du déploiement. Le nom de la VM suit le format vm-<username>.techvault.fr.

7.2 Intégration LDAP/Kerberos

Les rôles ldap_client, kerberos_client et kerberos_keytab configurent la VM pour que l'utilisateur puisse ouvrir une session graphique avec ses identifiants LDAP. Le keytab machine permet les montages NFS sécurisés avec sec=krb5.

7.3 Montages NFS automatiques

Point de montage	Source NFS	Droits	Description
/mnt/prive/<user>	nfs.techvault.fr:/srv/nfs/prive/<user>	chmod 700	Espace privé utilisateur
/mnt/commun	nfs.techvault.fr:/srv/nfs/partage	Lecture/Écriture	Espace partagé groupe

7.4 Supervision Wazuh

Le rôle wazuh-agent installe l'agent Wazuh sur les Vms contenant les services et sur chaque VM VDI pour centraliser les journaux système, détecter les anomalies d'authentification et surveiller l'intégrité des fichiers. Les alertes remontent au manager Wazuh de l'infrastructure.

8. Plan de Reprise Informatique (PRI)

Le PRI (Plan de Reprise Informatique) définit les procédures à appliquer en cas d'incident majeur afin de rétablir les services dans les délais les plus courts possibles. Les objectifs sont exprimés en DPO (Durée de Perte de Données Objective — RPO) et DRT (Durée de Reprise de Travail — RTO).

8.1 Tableau de synthèse

Scénario	DPO (RPO)	DRT (RTO)	Priorité
Perte serveur LDAP	24h	2h	🔴 Critique
Perte serveur NFS	24h	1h	🔴 Critique
Corruption base Kerberos	24h	1h	🔴 Critique
Perte Proxmox	24h	4h	🔴 Critique
Perte VM VDI	24h	30min	🟠 Majeur
Perte Flask / Nginx	0 (Git)	15min	🟡 Modéré
Perte configs IaC	0 (Git)	15min	🟡 Modéré

8.2 Procédure — Perte de l'un des serveurs

Procédure GUI commune (PBS)

1. Aller sur https://<IP_PROXMOX>:8006
2. Sélectionner nœud pve → Storage PBS → Contents
3. Identifier la sauvegarde par VM ID ou date
4. Cliquer sur Restore
5. Choisir Storage : local-lvm → cocher Unique
6. Suivre la progression dans Tasks
7. Démarrer la VM → vérifier le service

8.3 Amélioration du DPO

Amélioration	DPO cible	Moyen
Rsync toutes les heures	1h	cron */1 * * * *
Réplication LDAP	Temps réel	OpenLDAP
Snapshots PBS toutes les 6h	6h	Planification Proxmox Backup Server

9. Tests et validation

9.1 Tests de provisionnement OpenTofu

Test	Commande / Action	Résultat attendu
Init provider	tofu init	Provider telmate/proxmox téléchargé
Plan sans erreur	tofu plan	Plan affiché sans erreur
Création VM	tofu apply -auto-approve	VM créée, VMID = 2000+UID
IP attribuée	tofu output vm_ip	IP DHCP retournée
VMID correct	tofu output vm_id	VMID = 2000 + UID utilisateur
Clone complet	Interface Proxmox → VM → Hardware	Disque indépendant du template
Accès VNC	Proxmox → Console → VNC	Bureau Cinnamon visible

9.2 Tests de configuration Ansible

Test	Commande / Action	Résultat attendu
Ping Ansible	ansible -i inventory.ini vdi -m ping	pong retourné
Playbook sans erreur	ansible-playbook playbook.yml	0 failed
Hostname appliqué	hostname -f	vm-<user>.techvault.fr
DNS enregistré	nslookup vm-<user>.techvault.fr	IP de la VM retournée
Authentification LDAP	id <username>	uid=XXXX retourné
Montage NFS privé	df -h | grep /mnt/prive	Montage présent
Montage NFS commun	df -h | grep /mnt/partage	Montage présent
Agent Wazuh actif	systemctl status wazuh-agent	active (running)

9.3 Tests de l'interface Flask

Route	Test	Résultat attendu
/signup	Création compte avec email valide	Compte LDAP + Kerberos + NFS créés
/login	Authentification avec compte créé	Redirection vers /dashboard
/guest	Accès invité avec email	Compte temporaire 7 jours créé
/dashboard	Accès sans session	Redirection vers /login
/signup	Email déjà existant	Message d'erreur affiché

10. Sécurité

10.1 Mesures mises en œuvre

Domaine	Mesure	Détail
Réseau	VLAN VDI dédié	vmbr1 tag 5 — isolation des VMs
SSH	Clé dédiée www-data	/var/lib/www-data/.ssh/id_rsa
API Proxmox	Token API	Pas de mot de passe en clair
LDAP	Groupes séparés	etudiants (GID 5001) / guests (GID 5002)
Invités	Expiration automatique	shadowExpire = J+7
NFS	Permissions strictes	chmod 700 + chown uid:gid par utilisateur
Supervision	Agent Wazuh sur chaque VM	Centralisation logs + détection anomalies
Web	HTTPS via Nginx	Chiffrement des communications
Kerberos	Keytab machine	NFS sécurisé sec=krb5

10.2 Points d'amélioration

- Stocker token_secret dans une variable d'environnement (TF_VAR_token_secret)
- Ajouter terraform.tfvars et terraform.tfstate au .gitignore

Annexe — Flux complet de provisionnement VDI

```
Utilisateur
    ↓
[Interface Web Flask / Nginx HTTPS]
    │
    ├──1. Réception formulaire (nom + email)
    ├── 2. Calcul UID/GID depuis LDAP
    │
    ├──3. tofu apply → API Proxmox
    │     └─ Clone template Modele-Debian-cloudinit
    │     └─ VM démarrée, IP DHCP, cloud-init
    ├── 4. Récupération IP + VMID (tofu output)
    ├── 5. Génération inventaire Ansible (inventory.tpl → inventory.ini)
    ├── 6. ansible-playbook
    │     ├── set_hostname
    │     ├──dns_client + dns_register
    │     ├──ldap_client + kerberos_client + kerberos_keytab
    │     ├──nfs_client
    │     ├──create_user
    │     ├──wazuh-agent
    │     ├──Sécurisation SSH
    │     └─ Reboot VM
    ├──7. Email à l'utilisateur
    │     ├─ Nom de la VM
    │     ├─ Login (email)
    │     ├ Mot de passe temporaire
    │     └─ Accès VNC
```